BUKU PANDUAN UJIAN & PRESENTASI

Panduan Presentasi UTS Praktikum Laravel

Penjelasan Alur End-to-End: MVC, Database Migration, Eloquent Model, Seeder, dan Perintah Eksekusi Output

Studi Kasus: Sistem Perpustakaan Digital (Digital Library) • **Topik:** Modul 5 Migration & Model Eloquent

🎯 Tujuan Dokumen Ini

Dokumen ini disusun khusus sebagai panduan lengkap ketika Anda diminta **maju presentasi di hadapan dosen penguji** saat ujian praktikum / UTS. Panduan ini mencakup: (1) *Alur pembuatan sistem dari nol sampai akhir* secara runut dan teoritis, (2) *Kumpulan perintah & kode untuk menampilkan output* di Terminal, Database MySQL, maupun Web Browser, serta (3) *Skrip berbicara (dialog presentasi)* dan *bocoran pertanyaan dosen beserta cara menjawabnya secara profesional*.

1. Pemahaman Konsep & Arsitektur MVC

Sebelum membuka kode, jelaskan terlebih dahulu arsitektur dasar yang digunakan oleh Laravel. Ini menunjukkan bahwa Anda paham konsep dasar dan bukan sekadar *copy-paste*.

> 🗣️ **SKRIP KALIMAT YANG DIUCAPKAN KE DOSEN:**
>
> *"Selamat pagi / siang Bapak/Ibu dosen penguji. Pada tugas modul 5 ini, saya membangun backend perpustakaan digital menggunakan framework Laravel yang berbasis pola arsitektur MVC (Model-View-Controller). Model merepresentasikan struktur data dan logika bisnis tabel, View menyajikan tampilan visual kepada pengguna, dan Controller menghubungkan permintaan pengguna dengan Model. Pada praktikum ini, fokus utama adalah pada pembuatan Database Migration sebagai cetak biru tabel, Model Eloquent dengan relasi antar entitas, serta Seeder untuk pengisian data awal."*

Komponen Laravel	Lokasi File / Folder	Peran dalam Studi Kasus Ini
Database Configuration	`.env` & `config/database.php`	Mengatur koneksi MySQL (`digital_library`)
Migration	`database/migrations/`	Cetak biru (DDL) pembentukan tabel MySQL secara terprogram
Model (Eloquent ORM)	`app/Models/`	Kelas PHP yang merepresentasikan tabel & mendefinisikan relasi
Seeder	`database/seeders/`	Mengisi dummy record otomatis ke tabel saat deploy/test
Routing & Dashboard	`routes/web.php` & `resources/views/`	Menampilkan data output secara visual di browser pengguna

2. Tahapan Pembuatan dari Awal Sampai Akhir

Jelaskan langkah-langkah kerja Anda secara kronologis sesuai urutan berikut:

Langkah 1: Konfigurasi Database di file `.env`

Sebelum migrasi dijalankan, kita menyiapkan database di MySQL dan menghubungkannya di file `.env`:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=digital_library
DB_USERNAME=root
DB_PASSWORD=
```

> 🗣️ **PENJELASAN KE DOSEN:**
>
> *"Langkah pertama, saya membuat database MySQL bernama **digital_library**. Kemudian di file **.env**, saya ubah DB_CONNECTION menjadi mysql dan menyamakan nama DB_DATABASE. Laravel menggunakan file environment ini agar konfigurasi kredensial terpisah dari source code utama."*

Langkah 2: Pembuatan File Migration (Memperhatikan Urutan Dependency)

Migration dibuat menggunakan perintah artisan. **Poin penting yang sangat diperhatikan dosen adalah urutan pembuatan file** karena adanya *Foreign Key Constraint*.

Urutan	Perintah Artisan	Nama Tabel	Keterangan Ketergantungan (Dependency)
1	`php artisan make:migration create_members_table`	`members`	Tabel Master (Independen)
2	`php artisan make:migration create_authors_table`	`authors`	Tabel Master (Independen)
3	`php artisan make:migration create_publishers_table`	`publishers`	Tabel Master (Independen)
4	`php artisan make:migration create_books_table`	`books`	Bergantung pada `authors` & `publishers`
5	`php artisan make:migration create_loans_table`	`loans`	Bergantung pada `members` & `books`

> 💡 **PERTANYAAN JEBAKAN DOSEN:**
>
> **Dosen:** *"Kenapa tabel authors dan publishers harus dibuat SEBELUM tabel books?"*
> **Jawaban Anda:** *"Karena tabel books memiliki Foreign Key `author_id` dan `publisher_id` yang mereferensikan primary key dari tabel authors dan publishers. Jika tabel books dimigrasi lebih dulu sebelum tabel referensinya ada di MySQL, maka database akan melempar error `Foreign Key Constraint is incorrectly formed`."*

Detail Isi Kode Migration:

1. create_books_table.php:

```
Schema::create('books', function (Blueprint $table) {
    $table->id();
    $table->string('title');
    $table->foreignId('author_id')->constrained()->onDelete('cascade');
    $table->foreignId('publisher_id')->constrained()->onDelete('cascade');
    $table->timestamps();
});
```

Keterangan: Method `constrained()` secara otomatis mendeteksi tabel referensi berdasarkan konvensi nama kolom (contoh: `author_id` merujuk ke `authors.id`). Opsi `onDelete('cascade')` memastikan jika data author dihapus, semua buku karyanya otomatis ikut terhapus demi integritas data.

2. create_loans_table.php:

```
Schema::create('loans', function (Blueprint $table) {
    $table->id();
    $table->foreignId('member_id')->constrained()->onDelete('cascade');
    $table->foreignId('book_id')->constrained()->onDelete('cascade');
    $table->date('loan_date');
    $table->date('return_date')->nullable(); // nullable karena saat meminjam buku belum dikembalikan
    $table->timestamps();
});
```

Langkah 3: Pembuatan Model Eloquent & Relasi Antartabel

Model dibuat dengan perintah `php artisan make:model <NamaModel>`. Di dalam model, kita mendefinisikan properti `$fillable` untuk keamanan serta relasi antar tabel (One-to-Many).

Model	File	Relasi yang Didefinisikan	Kode Relasi
Author	`app/Models/Author.php`	Satu Author menulis banyak Buku (`hasMany`)	`public function books() { return $this->hasMany(Book::class); }`
Publisher	`app/Models/Publisher.php`	Satu Publisher menerbitkan banyak Buku (`hasMany`)	`public function books() { return $this->hasMany(Book::class); }`
Book	`app/Models/Book.php`	Buku milik Author & Publisher (`belongsTo`)	`public function author() { return $this->belongsTo(Author::class); }` `public function publisher() { return $this->belongsTo(Publisher::class); }`
Member	`app/Models/Member.php`	Satu Anggota memiliki banyak Transaksi (`hasMany`)	`public function loans() { return $this->hasMany(Loan::class); }`
Loan	`app/Models/Loan.php`	Transaksi merujuk ke Anggota & Buku (`belongsTo`)	`public function member() { return $this->belongsTo(Member::class); }` `public function book() { return $this->belongsTo(Book::class); }`

Langkah 4: Pembuatan Seeder & DatabaseSeeder

Seeder digunakan untuk memasukkan data awal dummy ke dalam tabel tanpa harus menginput manual lewat form.

- `MemberSeeder.php`: Memasukkan data anggota Ali dan Budi.
- `AuthorSeeder.php`: Memasukkan data penulis Tere Liye dan Andrea Hirata.
- `PublisherSeeder.php`: Memasukkan data penerbit Gramedia dan Mizan.
- `BookSeeder.php`: Memasukkan data buku Bumi (`author_id: 1` , `publisher_id: 1`) dan Laskar Pelangi (`author_id: 2` , `publisher_id: 2`).
- `LoanSeeder.php`: Memasukkan data transaksi peminjaman buku.

Semua seeder didaftarkan di dalam `DatabaseSeeder.php` :

```
class DatabaseSeeder extends Seeder {
    public function run(): void {
        $this->call([
            MemberSeeder::class,
            AuthorSeeder::class,
            PublisherSeeder::class,
            BookSeeder::class,
            LoanSeeder::class,
        ]);
    }
}
```

Langkah 5: Eksekusi Migrasi & Seeder Secara Sekaligus

Untuk menjalankan seluruh migrasi dari keadaan bersih dan langsung mengisinya dengan seeder, jalankan:

```
php artisan migrate:fresh --seed
```

> 🗣️ **PENJELASAN KE DOSEN:**
> *"Perintah `migrate:fresh --seed` akan menghapus seluruh tabel yang ada di database `digital_library` terlebih dahulu, menjalankan ulang seluruh file migrasi dari awal secara berurutan, lalu secara otomatis mengeksekusi kelas `DatabaseSeeder` untuk mempopulasikan data dummy ke dalam tabel."*

3. Kode-Kode & Perintah untuk Mengeluarkan Output

Saat presentasi, dosen sering berkata: "*Coba tunjukkan datanya keluar atau tidak!*". Tunjukkan kemampuan Anda dengan **3 cara berbeda** berikut:

Metode 1: Menampilkan Output Melalui Terminal CLI (Artisan Tinker)

Ini adalah metode tercepat dan paling disukai dosen karena menguji pemahaman Eloquent ORM di terminal.

A. Menampilkan Semua Buku Beserta Relasi Penulis & Penerbit (Eager Loading):

Cara 1: Masuk ke interactive shell:

```
php artisan tinker
>>> App\Models\Book::with(['author', 'publisher'])->get();
```

Cara 2: Eksekusi langsung (one-liner):

```
php artisan tinker --execute="echo json_encode(
    App\Models\Book::with(['author', 'publisher'])->get(),
    JSON_PRETTY_PRINT
);"
```

Hasil Output yang Tampil di Terminal:


```
[
  {
    "id": 1,
    "title": "Bumi",
    "author_id": 1,
    "publisher_id": 1,
    "author": { "id": 1, "name": "Tere Liye" },
    "publisher": { "id": 1, "name": "Gramedia" }
  },
  {
    "id": 2,
    "title": "Laskar Pelangi",
    "author_id": 2,
    "publisher_id": 2,
    "author": { "id": 2, "name": "Andrea Hirata" },
    "publisher": { "id": 2, "name": "Mizan" }
  }
]
```

B. Menampilkan Transaksi Pinjam Beserta Nama Peminjam & Judul Buku:

Cara 1: Masuk ke interactive shell:

```
php artisan tinker
>>> App\Models\Loan::with(['member', 'book'])->get();
```

Cara 2: Eksekusi langsung (one-liner):

```
php artisan tinker --execute="echo json_encode(
    App\Models\Loan::with(['member', 'book'])->get(),
    JSON_PRETTY_PRINT
);"
```

Hasil Output yang Tampil di Terminal:


```
[
  {
    "id": 1,
    "loan_date": "2026-10-01",
    "return_date": "2026-10-08",
    "member": { "id": 1, "name": "Ali", "email": "ali@mail.com" },
    "book": { "id": 1, "title": "Bumi" }
  },
  {
    "id": 2,
    "loan_date": "2026-10-05",
    "return_date": null,
    "member": { "id": 2, "name": "Budi", "email": "budi@mail.com" },
    "book": { "id": 2, "title": "Laskar Pelangi" }
  }
]
```

Metode 2: Menampilkan Output Melalui Web Browser (Live Dashboard)

Jika ingin presentasi yang sangat memukau secara visual, jalankan server Laravel dan buka dashboard yang sudah disiapkan.

Langkah Menjalankan Server:

```
php artisan serve
```

Buka browser dan akses alamat berikut:

- **Dashboard Visual:** `http://127.0.0.1:8000/` → Menampilkan kartu statistik total buku, total anggota, total peminjaman, serta tabel buku & status peminjaman lengkap dengan badge warna.
- **Output Raw Data JSON Buku:** `http://127.0.0.1:8000/api/books`
- **Output Raw Data JSON Pinjaman:** `http://127.0.0.1:8000/api/loans`

Metode 3: Menampilkan Output Melalui Database MySQL Langsung (SQL Query)

Jika dosen meminta dibukakan MySQL Command Line atau phpMyAdmin:

Perintah SQL JOIN untuk Membuktikan Relasi Foreign Key Bekerja:

```
mysql -u root
USE digital_library;

-- Query 1: Menampilkan Buku + Penulis + Penerbit
SELECT b.id, b.title, a.name AS Penulis, p.name AS Penerbit
FROM books b
JOIN authors a ON b.author_id = a.id
JOIN publishers p ON b.publisher_id = p.id;

-- Query 2: Menampilkan Data Peminjaman Lengkap
SELECT l.id, m.name AS Peminjam, b.title AS Judul_Buku, l.loan_date, l.return_date
FROM loans l
JOIN members m ON l.member_id = m.id
JOIN books b ON l.book_id = b.id;
```

4. Bocoran Tanya Jawab Kritis Dosen (Q&A Defense)

Berikut adalah pertanyaan yang hampir pasti diajukan oleh dosen penguji beserta jawaban terbaiknya:

Q1: Apa fungsi properti `$fillable` pada Model Eloquent?

Jawaban: Properti `$fillable` berfungsi sebagai proteksi terhadap celah keamanan *Mass Assignment Vulnerability*. Properti ini mendefinisikan kolom mana saja yang diizinkan untuk diisi secara massal saat menggunakan method seperti `Model::create($request->all())`. Jika tidak didefinisikan, Laravel akan melempar `MassAssignmentException` demi mencegah pengguna menyisipkan kolom berbahaya.

Q2: Apa perbedaan `php artisan migrate`, `migrate:rollback`, dan `migrate:fresh`?

Jawaban:

- `migrate`: Hanya mengeksekusi file migrasi baru yang belum pernah dijalankan (melihat tabel `migrations`).
- `migrate:rollback`: Membatalkan/menjalankan method `down()` pada batch migrasi terakhir.
- `migrate:fresh`: Menghapus (drop) seluruh tabel di database tanpa menjalankan `down()`, lalu membuat ulang seluruh tabel dari awal.

Q3: Apa yang dimaksud dengan N+1 Problem pada Eloquent dan bagaimana mengatasinya?

Jawaban: N+1 Problem terjadi ketika kita mengambil kumpulan data utama (1 query), lalu melakukan loop dan memanggil relasi di setiap baris sehingga memicu query baru sebanyak N kali (total 1 + N query). Solusinya adalah menggunakan teknik **Eager Loading** dengan method `with()`, misalnya `Book::with('author')->get()`, yang hanya membutuhkan 2 query SQL saja terlepas dari seberapa banyak jumlah buku yang diambil.

Q4: Apa arti `onDelete('cascade')` pada relasi Foreign Key?

Jawaban: Aturan ini memastikan integritas referensial. Apabila baris data induk (misalnya record Author) dihapus dari database, maka seluruh data anak yang berelasi dengannya (semua Buku yang ditulis oleh author tersebut) akan otomatis ikut dihapus oleh sistem database, sehingga tidak menyisakan data yatim (*orphan data*).

5. Ringkasan Checklist Presentasi di Depan Dosen

1. Pastikan service MySQL XAMPP sudah berjalan (hijau).
2. Buka terminal di folder proyek `d:\tugas`.
3. Siapkan 1 tab terminal untuk perintah tinker: `php artisan tinker`.
4. Siapkan 1 tab terminal untuk web server: `php artisan serve`.
5. Buka browser pada `http://127.0.0.1:8000/` untuk demonstrasi live dashboard.
6. Jelaskan secara tenang dengan alur: **Latar Belakang → Konfigurasi .env → Migration & Constraint → Model & Relasi → Seeder → Demo Output**.